Cost-Efficient Foreground Segmentation Using a Lightweight Depthwise-Separable U-Net under a Single-Core CPU Budget

Rahul Kumar Bhikundia
Roll Number: M26CSA006
CSL7360: Computer Vision — Assignment 2
Indian Institute of Technology Jodhpur

Abstract

This report describes a compact binary segmentation system that labels every pixel of a 256×256 photograph as main object or background, while staying within a single-core CPU budget of 50 ms per image and a small storage footprint. For Track B, a U-Net-style fully convolutional network, `SmallUNet`, was designed and trained from scratch on DUTS-TR, with no pretrained weights. It is built from depthwise-separable convolutions, batch normalization, bilinear upsampling and skip connections, with very narrow channel widths ($3 \to 8 \to 16 \to 32 \to 48$) and full 256×256 input resolution. The result is a network of $15,949$ trainable parameters that occupies $129,326$ bytes on disk (≈ 0.1233 MiB, about 3.08% of the 4 MiB reference size). On the 5,019 DUTS-TE test images it reaches mIoU $= 0.64133$ and BF1 $= 0.30448$, giving $Q = 0.54027$, at an average of 25.2291 ms per image on one CPU core. The resulting Cost-Efficient Segmentation (CES) score is 0.48408.

1 Objective

The goal is to build a binary segmentation pipeline that gives every pixel of a photograph a label (1 for the main object, 0 for background) and runs on a single CPU thread within a tight time budget, with the network trained entirely from scratch. Concretely, the pipeline has to:

- keep the average single-core CPU time at or below 50 ms per photograph over the 5,019 DUTS-TE images;
- use a small fully convolutional network (FCN) built from basic PyTorch layers and trained from scratch, with no pretrained weights or external segmentation libraries;
- score as high as possible on the Cost-Efficient Segmentation (CES) metric, which trades off region overlap (mIoU), boundary accuracy (Boundary F1), single-threaded CPU time (T_{CPU}) and parameter-file size (S_{disk});
- hold up on real campus and hostel scenes with varied lighting, object scale, clutter and partial occlusion.

2 Overall Pipeline Architecture

The pipeline has the following stages. The photograph is resized offline. The timed part then converts BGR to RGB and scales the values to $[0, 1]$. A three-stage depthwise-separable U-Net encoder ($8 \to 16 \to 32$ channels) feeds a 48-channel bottleneck at 32×32 resolution. A three-stage decoder uses bilinear upsampling and concatenated skip connections ($80 \to 32$, $48 \to 16$ and $24 \to 8$ channels), a 1×1 convolution produces one logit per pixel, and thresholding at zero gives

the mask. Figure 1 shows this flow; the timed part starts at the box labelled "Timed Pipeline Entry".

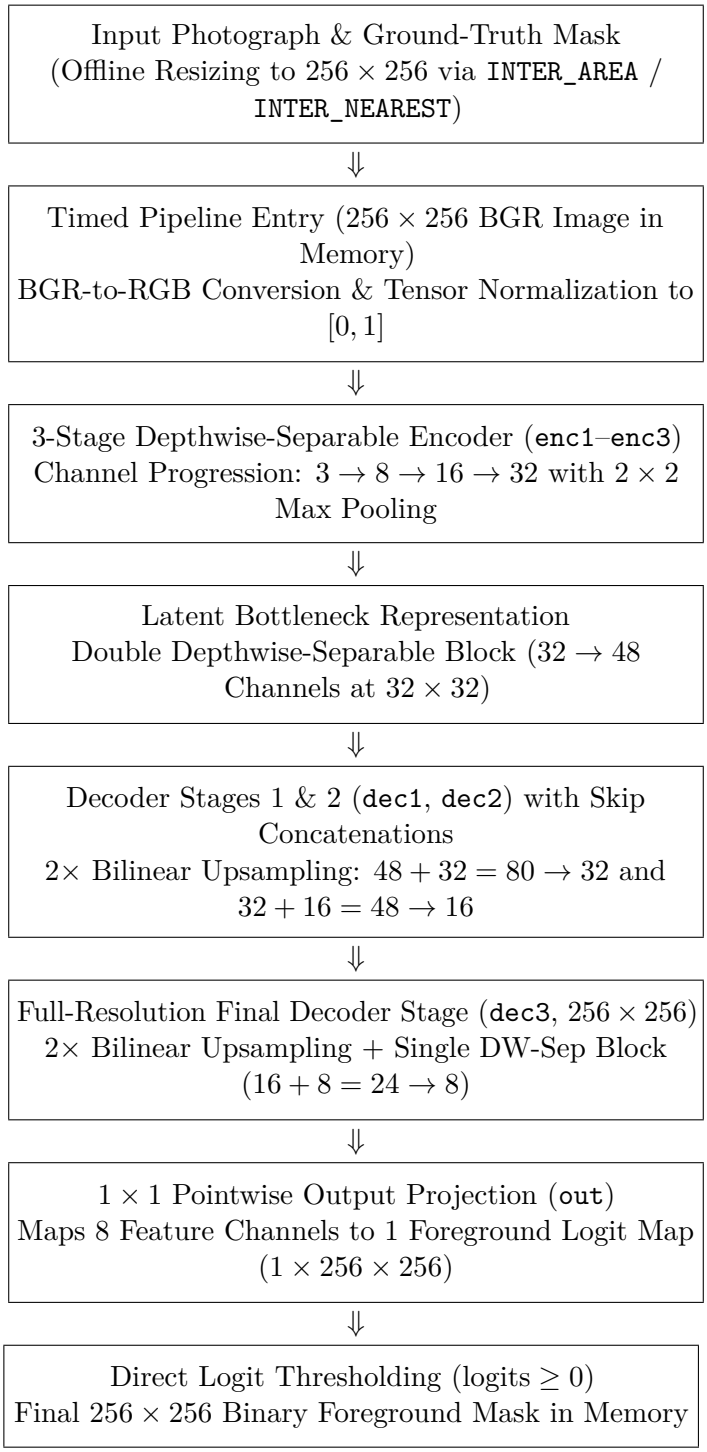


Figure 1: System architecture and timed execution dataflow of the `SmallUNet` segmentation pipeline.

3 Dataset and Preprocessing

3.1 DUTS Dataset Partitioning

DUTS is a salient-object dataset and supplies all training, validation and test images used here. To keep model selection separate from final evaluation, the 10,553 images of `DUTS-TR` were randomly split 90%/10% with a fixed seed (`torch.Generator().manual_seed(42)`). `DUTS-TE` was left untouched until the checkpoint had been frozen:

- **DUTS-TR training subset:** $9,497$ image–mask pairs; tensor shapes $(3, 256, 256)$ and $(1, 256, 256)$.
- **DUTS-TR validation subset:** $1,056$ pairs, used for tuning and for choosing the checkpoint.
- **DUTS-TE test set:** $5,019$ pairs, used only for the final evaluation and the CPU benchmark.

3.2 Image and Mask Resizing Protocol

Following Section 1b of the assignment, every image and mask is brought to 256×256 pixels:

1. Photographs are read with OpenCV and resized using area interpolation (`cv2.INTER_AREA`).
2. Masks are loaded as grayscale and resized with nearest-neighbour interpolation (`cv2.INTER_NEAREST`), which avoids creating intermediate gray values along the edges.
3. A mask pixel with value ≥ 128 is foreground (1.0); any lower value is background (0.0).
4. Images are converted from BGR to RGB, rearranged to channel-first $(3, 256, 256)$ and divided by 255.0 so that values lie in $[0, 1]$.

3.3 Personal Photograph Dataset (`captured/`)

To test generalization beyond DUTS, six photographs were taken around the campus and hostel. Together they cover indoor and outdoor scenes, daylight and tube light, near and far objects, cluttered backgrounds and partial occlusion. The script `prepare_captured.py` organizes them into `original/`, `images_256/` and `masks_256/` inside `captured/`, with matching file names `photo1` to `photo6`. The scenes are a potted plant (`photo1`), a matte black water bottle on a cluttered desk (`photo2`), a sneaker on an indoor floor (`photo3`), a biscuit packet partly hidden behind an air-conditioner remote control (`photo4`), a distant outdoor water dispenser (`photo5`) and a bicycle against a brick wall (`photo6`).

4 Proposed Network Architecture (`SmallUNet`)

The network must fit in 50 ms on one CPU core, yet it works at the full 256×256 resolution so that small objects and thin boundaries are not lost. For this purpose `SmallUNet`, a compact U-Net-style encoder–decoder, was written from scratch in PyTorch. A standard 3×3 convolution needs $9\, C_{\text{in}} C_{\text{out}} HW$ multiply-accumulate operations (MACs), so every convolution is instead split into spatial filtering and channel mixing, i.e. a **depthwise-separable convolution**:

1. **Depthwise convolution:** a 3×3 filter is applied to each input channel separately (padding $= 1$, groups $= C_{\text{in}}$), followed by BatchNorm2d(C_{in}) and an in-place ReLU.
2. **Pointwise convolution:** a 1×1 convolution mixes the channels ($C_{\text{in}} \to C_{\text{out}}$), again followed by BatchNorm2d(C_{out}) and an in-place ReLU.

Every encoder and decoder block (`ConvBlock`) stacks two such units. The last decoder stage (`FinalDecoderBlock`) runs at full resolution and uses only one ($24 \to 8$ channels at 256×256), followed by a 1×1 output convolution ($8 \to 1$), to keep the most expensive resolution cheap. Limiting the channel widths to $\{8, 16, 32, 48\}$ leaves only $15,949$ trainable parameters ($129,326$ bytes on disk). Table 1 lists each stage with its parameter count.

Table 1: Layer-by-layer specification and trainable parameter breakdown of `SmallUNet`.

Stage	Block / Operation	Channels ($C_{\text{in}} \to C_{\text{out}}$)	Spatial Resolution	Parameters
`enc1 + pool1`	`ConvBlock` (2× DW-Sep) + 2×2 MaxPool	$3 \to 8$	$256 \times 256 \to 128 \times 128$	268
`enc2 + pool2`	`ConvBlock` (2× DW-Sep) + 2×2 MaxPool	$8 \to 16$	$128 \times 128 \to 64 \times 64$	768
`enc3 + pool3`	`ConvBlock` (2× DW-Sep) + 2×2 MaxPool	$16 \to 32$	$64 \times 64 \to 32 \times 32$	2,304
`bottleneck`	`ConvBlock` (2× DW-Sep)	$32 \to 48$	32×32	5,088
`up1 + dec1`	2× Bilinear + Concat(`enc3`) + `ConvBlock`	$48 + 32 = 80 \to 32$	$32 \times 32 \to 64 \times 64$	5,120
`up2 + dec2`	2× Bilinear + Concat(`enc2`) + `ConvBlock`	$32 + 16 = 48 \to 16$	$64 \times 64 \to 128 \times 128$	1,888
`up3 + dec3`	2× Bilinear + Concat(`enc1`) + `FinalDecoderBlock`	$16 + 8 = 24 \to 8$	$128 \times 128 \to 256 \times 256$	504
`out`	1×1 Pointwise Output Convolution	$8 \to 1$	256×256	9
Total	**Complete Trainable Network**	—	$\mathbf{256 \times 256 \to 256 \times 256}$	**15,949**

5 Training Configuration and Checkpoint Selection

`SmallUNet` was trained from scratch on the 9,497-image training split using an NVIDIA GTX 1650 GPU. Training ran for up to 50 epochs with early stopping (patience 7), a batch size of 16, the Adam optimizer (lr = 0.001) and a pixel-wise binary cross-entropy loss with logits (`nn.BCEWithLogitsLoss()`):

$$\mathcal{L}_{\text{BCE}}(\hat{z}, y) = -\frac{1}{N} \sum_{i=1}^{N} \Big[y_i \log \sigma(\hat{z}_i) + (1 - y_i) \log \big(1 - \sigma(\hat{z}_i)\big) \Big] \tag{1}$$

Here $\hat{z}_i$ is the predicted logit at pixel i, $y_i \in \{0, 1\}$ is the ground-truth label and $\sigma(\hat{z}_i) = (1+e^{-\hat{z}_i})^{-1}$. At inference a pixel is labelled foreground when $\hat{z}_i \geq 0$, which is equivalent to $\sigma(\hat{z}_i) \geq 0.5$ and saves computing a sigmoid for all $65,536$ pixels.

After every epoch the loss was measured on the 1,056-image validation set. It fell from 0.4087 in epoch 1 (training loss 0.4617) to a minimum of 0.2603 in epoch 44 (training loss 0.2500). The checkpoint with the lowest validation loss was saved as `checkpoints/best_model_4.pth` ($129,326$ bytes) and kept fixed for all final evaluations. Table 2 summarizes the setup.

Table 2: Summary of training hyperparameters and checkpoint selection.

Configuration Parameter	**Value / Specification**
Framework	PyTorch (Track B FCN from scratch)
Training / Validation Split	$9,497$ Training / $1,056$ Validation (Seed = 42)
Input / Operating Resolution	256×256 Full-Resolution Forward Pass
Batch Size / Max Epochs	16 / 50 Epochs (Early Stopping Patience = 7)
Optimizer / Learning Rate	Adam / 1.0×10^{-3}
Loss Function	Binary Cross-Entropy with Logits (`BCEWithLogitsLoss`)
Validation Loss Trajectory	0.4087 (Epoch 1) $\to$ 0.2603 (Epoch 44, Best)
Training Hardware	NVIDIA GeForce GTX 1650 GPU
Timing Hardware (CPU)	11th Gen Intel Core i5-11400H @ 2.70 GHz, 7.8 GB RAM
Saved Parameter File	`checkpoints/best_model_4.pth` ($129,326$ bytes)

6 Evaluation Metrics

6.1 Mean Intersection over Union (mIoU)

Region overlap is computed from one global 2×2 confusion matrix accumulated over all images in a set, with true positives (TP), false positives (FP), false negatives (FN) and true negatives (TN). The IoU is taken separately for background and foreground, and mIoU is their average:

$$\mathrm{IoU}_{\mathrm{bg}} = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP} + \mathrm{FN}}, \qquad \mathrm{IoU}_{\mathrm{fg}} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}, \qquad \mathrm{mIoU} = \frac{\mathrm{IoU}_{\mathrm{bg}} + \mathrm{IoU}_{\mathrm{fg}}}{2} \tag{2}$$

6.2 Boundary F1-Score (BF1)

Boundary quality is measured with BF1. A foreground pixel is a boundary pixel if at least one of its four neighbours (above, below, left, right) is background, and pixels outside the 256×256 frame count as background. A predicted boundary pixel is correct if a true boundary pixel lies in its 5×5 window, and a true boundary pixel is found if a predicted boundary pixel lies in its 5×5 window. In `evaluate.py` and `evaluation_test.py`, both the boundary extraction (including foreground pixels on the image border) and the 5×5 window dilation use vectorized NumPy shifts of boolean arrays instead of per-pixel loops. Precision P, recall R and the per-image BF1 are:

$$P = \frac{\text{Matched Predicted Boundary Pixels}}{\text{Total Predicted Boundary Pixels}}, \qquad R = \frac{\text{Matched True Boundary Pixels}}{\text{Total True Boundary Pixels}} \tag{3}$$

$$\mathrm{BF1} = \frac{2PR}{P + R} \tag{4}$$

The per-image BF1 is then averaged over all images in the set.

6.3 Quality Score (Q) and Cost-Efficient Segmentation (CES) Score

The quality score Q and the Cost-Efficient Segmentation (CES) score are defined as:

$$Q = 0.70\,\mathrm{mIoU} + 0.30\,\mathrm{BF1} \tag{5}$$

$$\mathrm{CES} = \begin{cases} Q\left[1 - 0.20\,\dfrac{T_{\mathrm{CPU}}}{50\ \mathrm{ms}} - 0.10\,\min\left(1, \dfrac{S_{\mathrm{disk}}}{4\ \mathrm{MiB}}\right)\right] & \text{if } T_{\mathrm{CPU}} \le 50\ \mathrm{ms} \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

where $4\ \mathrm{MiB} = 4,194,304$ bytes.

7 Single-Threaded CPU Benchmarking Protocol

Single-thread execution is enforced in both `benchmark.py` and `predict.py`. The environment variables `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS` and `VECLIB_MAXIMUM_THREADS` are set to `1` before any numerical library is imported, and PyTorch and OpenCV are limited with `torch.set_num_threads(1)`, `torch.set_num_interop_threads(1)` and `cv2.setNumThreads(1)`. When the model is loaded (outside the timed region), the checkpoint is traced and optimized with TorchScript (`torch.jit.optimize_for_inference`), which folds each `BatchNorm2d` into the preceding `Conv2d`; this changes the arithmetic only at floating-point rounding level. After 10 warm-up runs, each pre-resized 256×256 image is timed individually with `time.process_time()`, covering BGR-to-RGB conversion, tensor preparation, the forward pass and conversion to a binary mask. All timings were taken on an 11th Gen Intel Core i5-11400H CPU (2.70 GHz) with 7.8 GB RAM running Windows, on a single thread. On Windows, `process_time()` advances in ticks of 15.625 ms, so every individual measurement, and therefore the median and maximum, is a multiple of that value; the average over $5,019$ images is the meaningful figure. Repeated runs of `benchmark.py` with the final files gave averages of 24.55–25.23 ms.

8 Experimental Results

Table 3 gives the segmentation results on the DUTS-TR validation split, the 5,019-image DUTS-TE test set and the six captured photographs. Table 4 gives the single-core CPU timing and storage figures measured on DUTS-TE.

Table 3: Overall segmentation quality and cost-efficiency metrics across all evaluation datasets.

Evaluation Split	**Images**	**BG IoU**	**FG IoU**	**mIoU**	**BF1**	**Quality (Q)**	**T_{CPU} (ms)**	**S_{disk} (B)**	**CES**
DUTS-TR Validation	1,056	0.85649	0.65188	0.75418	0.40062	0.64812	—	129,326	—
DUTS-TE Official Test	5,019	0.85535	0.42731	0.64133	0.30448	0.54027	25.2291	129,326	**0.48408**
Captured Photos	6	0.90394	0.53438	0.71916	0.38660	0.61939	28.12	129,326	—

Table 4: Official single-threaded CPU benchmark summary across all 5,019 DUTS-TE test images.

Benchmark Measurement	**Measured Result**
Total Test Images Processed (`DUTS-TE`)	5,019
Average CPU Inference Time (T_{CPU})	25.2291 ms
Median CPU Inference Time*	31.2500 ms
Maximum CPU Inference Time*	62.5000 ms
Saved Checkpoint Size (S_{disk})	129,326 bytes ($\approx$ 0.1233 MiB)
Time Penalty Factor ($0.20 \times T_{CPU}/50$ ms)	0.10092
Size Penalty Factor ($0.10 \times S_{disk}/4$ MiB)	0.00308
Final Cost-Efficient Segmentation (CES) Score	**0.48408**

*Quantized to multiples of the 15.625 ms Windows `process_time` tick.

Inserting the DUTS-TE values ($Q = 0.54027$, $T_{CPU} = 25.2291$ ms ≤ 50 ms and $S_{disk} = 129,326$ bytes) into the CES formula gives:

$$\text{CES} = 0.54027 \times \left(1 - 0.20 \times \frac{25.2291}{50.0} - 0.10 \times \frac{129,326}{4,194,304}\right) = 0.54027 \times 0.89600 = 0.48408 \quad (7)$$

9 Personal Photograph Evaluation and Visualizations

Table 5 gives the Boundary F1 and the `predict.py` CPU time for each of the six captured photographs. For every photograph, `visualize.py` produces a four-panel figure: the original image, the ground truth as a semi-transparent red overlay, the prediction as a semi-transparent green overlay, and an agreement map in which yellow marks correctly found foreground, red marks missed foreground and green marks false foreground. The figures are shown in Figures 2 and 3.

Table 5: Per-image evaluation metrics across the six captured campus and hostel photographs.

Photo ID	**Object**	**Scene & Environmental Conditions**	**Boundary F1**	**`predict.py` T_{CPU} (ms)**
`photo1`	Plant	Outdoor steps, daylight, near, textured wall	0.4340	35.16
`photo2`	Bottle	Indoor desk, tube light, near, cluttered desk	0.3017	27.34
`photo3`	Shoe	Indoor floor, tube light, near, clear silhouette	0.7565	26.56
`photo4`	Occluded Packet	Indoor close-up, partial occlusion by controller	0.2869	27.34
`photo5`	Water Dispenser	Outdoor steps, daylight, far/small object	0.1570	24.22
`photo6`	Bicycle	Outdoor brick wall, daylight, thin spokes & shadow	0.3835	28.12
Aggregate	**All 6 Photos**	**Global mIoU** = 0.71916**, BG IoU** = 0.90394**, FG IoU** = 0.53438	**0.38660**	**Q = 0.61939**

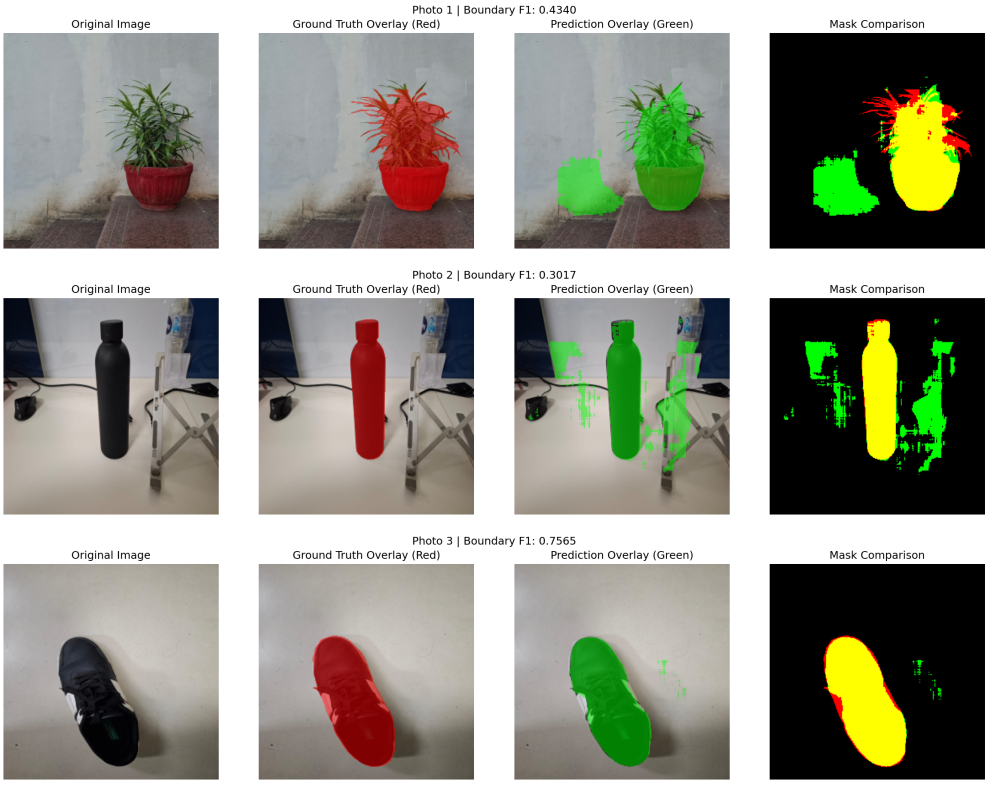


Figure 2: Four-panel qualitative segmentation comparisons for `photo1` (plant, BF1 = 0.4340), `photo2` (water bottle, BF1 = 0.3017), and `photo3` (shoe, BF1 = 0.7565).

10 Discussion and Failure Case Analysis

Validation versus test gap: Q falls from 0.648 on the validation split to 0.540 on DUTS-TE, and foreground IoU from 0.652 to 0.427. The validation images come from the same DUTS-TR distribution as the training data, whereas DUTS-TE is likely harder, with more clutter and small objects that a 15,949-parameter network without semantic priors handles poorly. The training and validation losses are close (0.2500 and 0.2603), so the model appears limited by capacity rather than by overfitting.

Good cases (`photo3`, `photo1`): The shoe in `photo3` has the best boundary score (BF1 = 0.7565) because the dark shoe stands out strongly against the plain, evenly lit floor. The plant in `photo1` (BF1 = 0.4340) is segmented across the pot and most of the foliage, but fine leaf tips are missed and a large false region appears on the dark-stained lower-left wall and the reddish step, probably because their colour resembles the red pot.

Failure case 1 — small distant object in a cluttered scene (`photo5`, BF1 = 0.1570): The water dispenser covers only 1.23% of `photo5`. The network still captures most of the purple cooler body (yellow on the agreement map), but high-contrast structures in the background, such as the vertical pillar edges and reflections on the steps, produce false-positive regions (green). These are large compared with the tiny true object, which explains the low overlap and the low BF1.

Failure case 2 — partial occlusion (`photo4`, BF1 = 0.2869): In `photo4` the purple biscuit packet is partly covered by a bright white air-conditioner remote control with a high-contrast LCD and a circular button pad. The model finds most of the visible packet (yellow) but misses part of its lower right side (red), and it also fires on the LCD, the button pad and the nearby edge of the remote (green). Without semantic category priors, the network has no way to tell that the remote is the occluder and not the target.

Failure case 3 — desk clutter and cast shadows (`photo2`, BF1 = 0.3017; `photo6`,

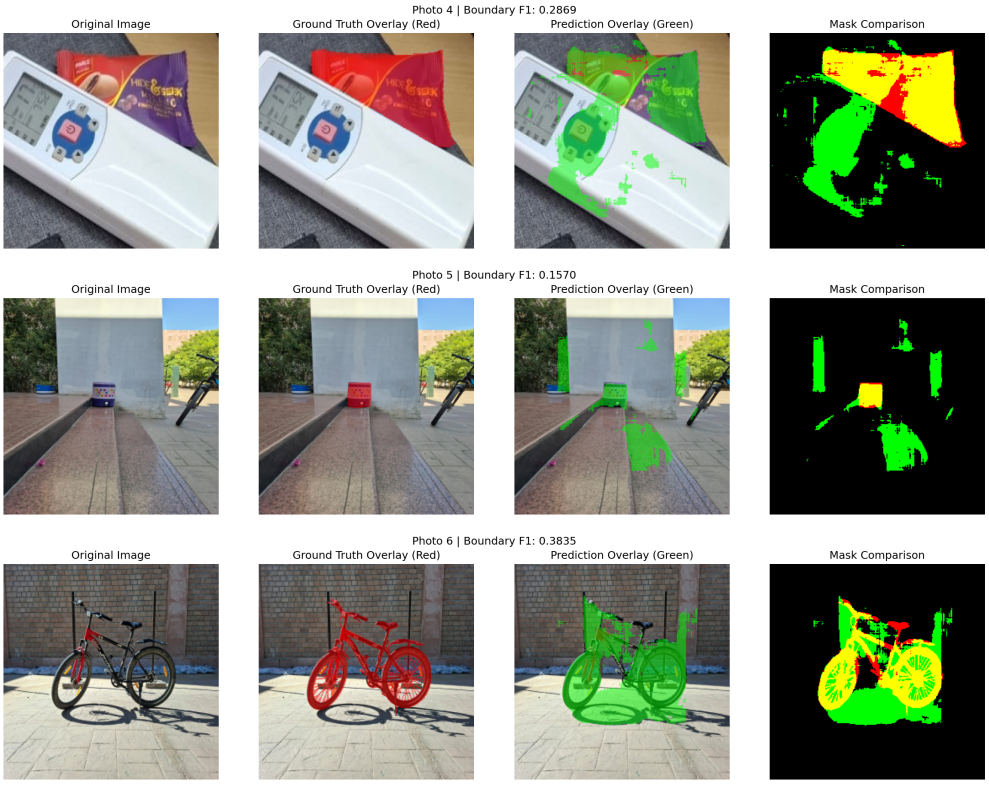


Figure 3: Four-panel qualitative segmentation comparisons for `photo4` (occluded packet/controller, BF1 = 0.2869), `photo5` (distant water dispenser, BF1 = 0.1570), and `photo6` (bicycle, BF1 = 0.3835).

BF1 = 0.3835): In `photo2` the black bottle is recovered almost completely, with hardly any missed pixels, but clutter on the desk (the metallic laptop stand, the plastic bottle behind it and the dark wall panel) causes false positives. In `photo6` the bicycle frame and wheels are found, but the strong shadow on the sunlit pavement is labelled as foreground. The largest false region there is the brick wall behind and between the wheels and the neighbouring pillar, probably because its dark texture resembles the dark bicycle frame.

11 Conclusion

A lightweight foreground-segmentation network, `SmallUNet`, was designed, trained from scratch on DUTS-TR for 50 epochs and evaluated under a single-core CPU budget. Narrow channel widths ($3 \to 8 \to 16 \to 32 \to 48$) and depthwise-separable convolutions at 256×256 resolution give a model with $15,949$ parameters ($129,326$ bytes) that reaches mIoU = 0.64133, BF1 = 0.30448, $Q = 0.54027$ and CES = 0.48408 on the 5,019-image DUTS-TE benchmark.